Datasheet BOLT - 16bit µProcesseur

Binary Operations Logic Translator

Conçus par **SLIMANI Yanis**
Etudiant en Licence 3 d'Électronique, énergie électrique, automatique
Sorbonne Université

Mail : Yanis.Slimani@etu.sorbonne-universite.fr

Description du Projet : BOLT-16

Le projet BOLT-16 consiste en la conception intégrale, from scratch, d'une architecture de microprocesseur 16 bits. L'objectif est de s'approprier l'ensemble du cycle de conception d'un système numérique, en développant non seulement le cœur de calcul (CPU), mais aussi des interfaces de communication.

Objectifs du Projet (Liste non exhaustive)

1. Conception de l'Unité de Contrôle :

- Développement d'une Machine à États (MAE) gérant le cycle de vie du processeur : *Boot, Sleep, Fetch, Decode, Execute* et *Update PC*.
- Réalisation d'un Décodeur d'instructions propriétaire capable d'orchestrer les signaux de commande du système.

2. Développement de la Partie Opérative (ALU) :

- Implémentation d'une ALU 16 bits supportant 16 opérations arithmétiques et logiques distinctes.
- Gestion précise du timing des registres : Registre d'Instruction (IR), Compteur de Programme (PC), Registre d'Adresse (ADR) et l'Accumulateur (Acc).

3. Interface et Gestion des I/O (GPIO) :

- Création d'un système de Memory-Mapped I/O via deux registres d'interface dédiés.
- Affectation d'adresses spécifiques pour le contrôle physique : `xFFE` pour la lecture (Input) et `xFFF` pour l'écriture (Output).

4. Architecture de Boot :

- Mise en œuvre d'une liaison UART entre la mémoire d'instructions (ROM/RAM) et un bloc extérieur.
- Objectif est de permettre le téléchargement dynamique du code machine.

5. Application externe:

- Développement d'une application Python dédiée servant de compilateur (Pseudo code vers Assembleur vers Binaire).
- Example d'OS en pseudo code.

6. Méthodologie et Analyse Personnelle :

- Démarche de test itérative par validation de chaque bloc et test de chaque instruction à chaque étape d'ajout.
- Travail de réflexion architecturale personnelle, conception originale.
- Production d'une documentation technique détaillée et de schématisations fonctionnelles.

Footprint

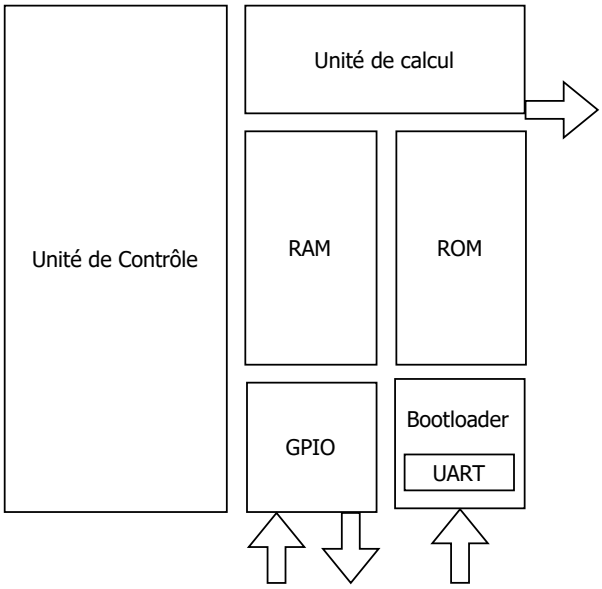
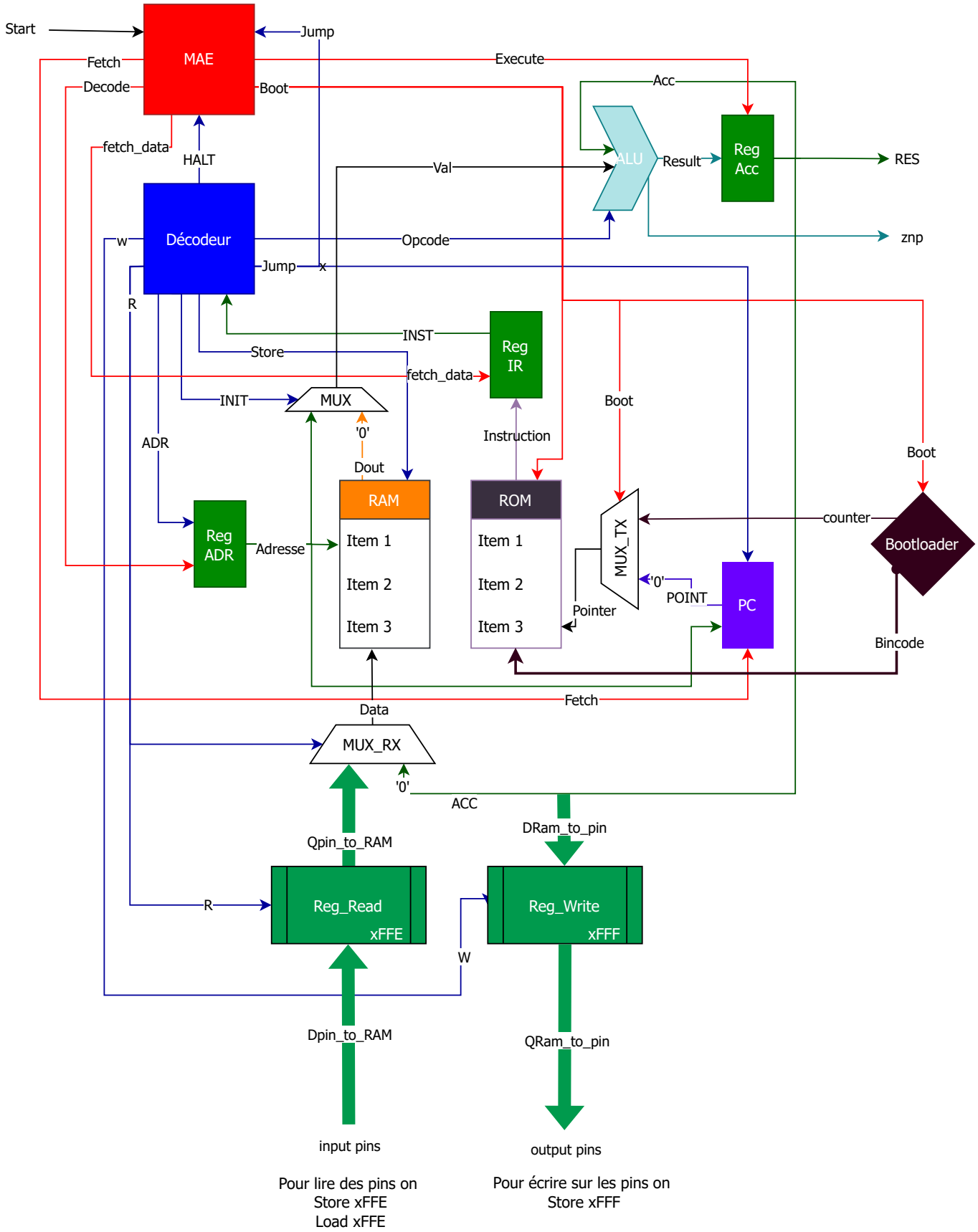


Schéma fonctionnelle



Description par bloc dans l'ordre d'exécution

1.Bootloader

Le bootloader en soit est un bloc complexe indépendant, rapidement pendant l'état de boot, la ROM recoit l'adressage depuis le bloc boot ainsi que les donnés a écrire dessus (reçus depuis un bloc extérieur en communication UART.

Protocol de Communication :

1	1	0	Taille du code 2 octet	1	1	0	Instruction 2 octet	1	1	0	Instruction 2 octet	1

Le système est organiser autour d'une machine a état, dont la fonction est d'attendre le start bit (interruption), compter jusqu'à 8 (acquissions du premier octet) tout en décalant les bits reçus dans un registre a décalage.

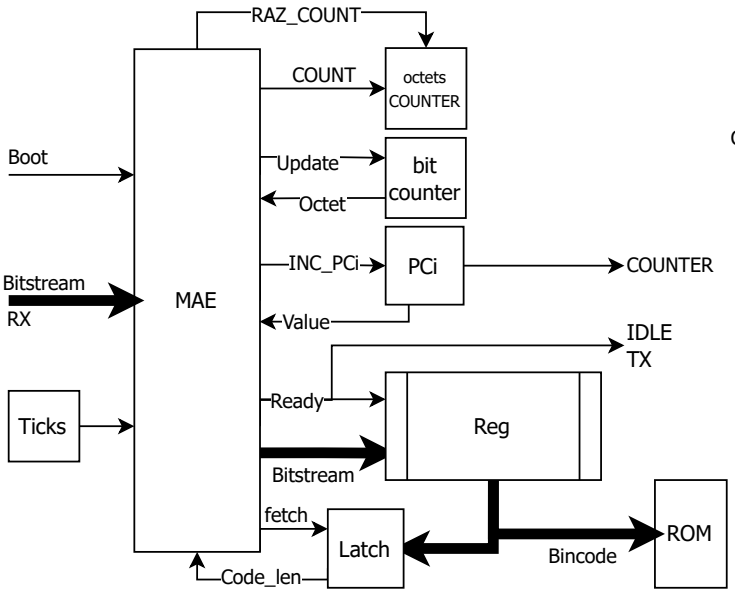
En pratique ca suffirait mais nous travaillons sur des instruction sur 16bit, donc en doit acquérir 2 octets par instruction, pour cela en incrémente un compteur (octets counter), en le remet a zéro des que 2 octet en était acquis, pour préparer la prochaine instruction.

La trame du bitstream sera organiser de cette façon, un IDLE a '1', un start bit '0' et un stop bit '1' :

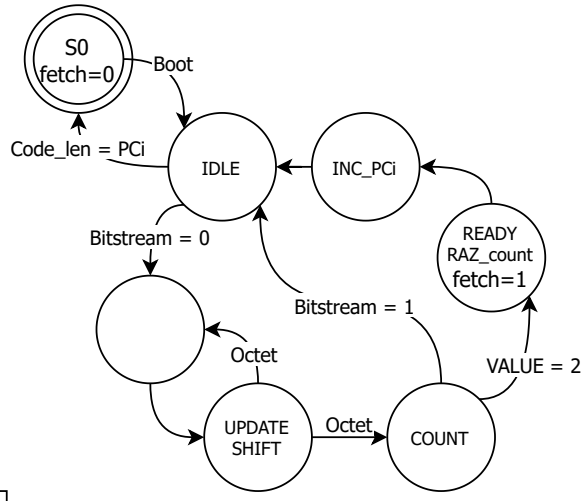
1	1	1	Start bit	D0	D1	D2	D3	D4	D5	D6	D7	Stop bit	1	1	1

T0 — T1 — T2 — T3 — T4 — T5 — T6 — T7 — T8 — T9 →

Schéma Bloc interne



Graphe d'état



State	Explication
S0	Etat de remise a zéro
IDLE	Attende d'une interuption
NC	Etat NULL
UPDATE_SHIFT	Incrémente le compteur de bit reçu, et les shift dans le registre
COUNT_octets	compte le nombre d'octets reçu, les instruction sont sur deux octets
INST_Ready	Boucle 0 : Latch de la taille du code Boucle i : Latch des octets reçus pour l'écriture sur ROM, remise a zéro du compteur
INC_PCi	Incrémente PCi, l'adresse de la prochaine instruction a transmettre

Tick rate generator

Le Tick Rate Generator est un compteur dont la fonction est de lever un signal `tick` à '1' afin d'indiquer au bootloader le moment précis où nous lisons le milieu du bit reçu. Cette méthode permet de capturer la donnée à l'instant où elle est la plus stable sur la ligne.

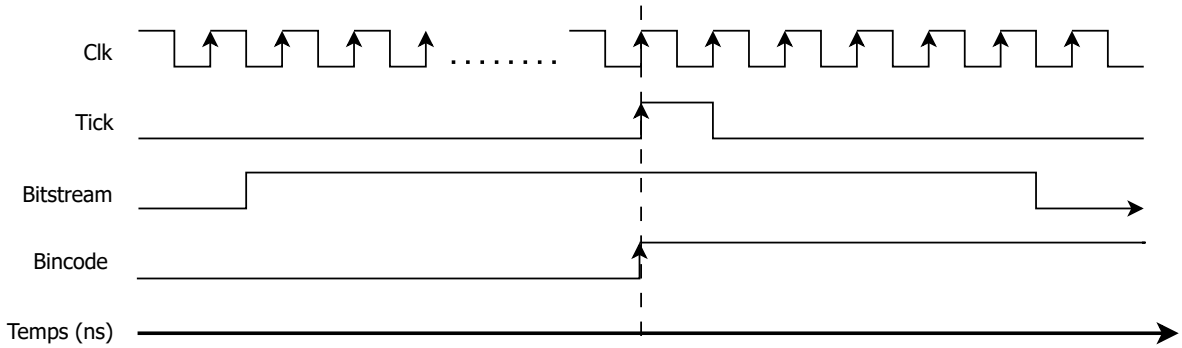
- Débit du bitstream : 9600 bits/seconde (9600 Bauds).
- Fréquence CPU (simulation) : 50 MHz.
- Durée d'un bit : 5208 coups d'horloge (50M/ 9600).

Pour lire chaque bit en son milieu, on génère un signal `ENABLE = tick` qui s'active selon la règle temporelle suivante :

1. Un premier délai de 2604 coups (demi-bit) après le début du Start Bit.
2. Puis une répétition tous les 5208 coups d'horloge (un bit complet).

L'échantillonnage se fait donc à 2604 + n*5208 coups d'horloge (où n est l'index du bit).

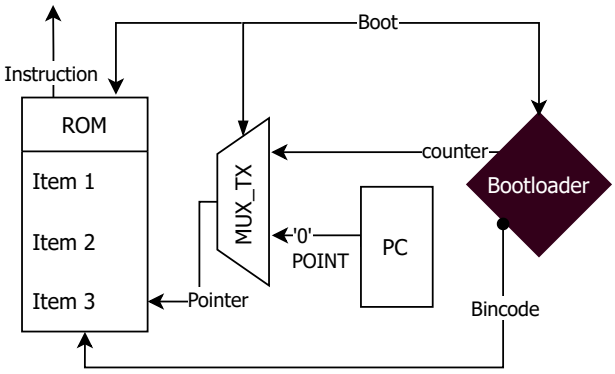
Un chronogramme illustratif ci-dessous détaille cette synchronisation.



Fonctionnement

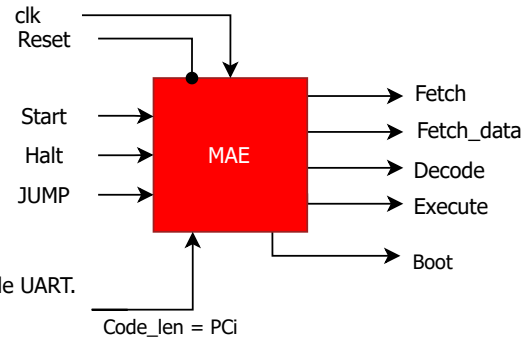
Au démarrage du CPU, Boot est a 1 pour télécharger le code a exécuté puis repasse a 0.

Boot = 1	Boot = 0
ROM est piloté par le Bootloader, qui indique l'adresse, la donnés a écrire dessus (Bincode).	ROM est piloté par PC (durant l'exécution du program) et aiguille les instructions
Mode Ecriture	Mode Lecture



2.Machine a état

- C'est le chef d'orchestre. Elle séquence le fonctionnement du processeur en basculant entre les différents états (Boot, Sleep, Fetch, Decode, Execute, Update_PC).
- **Ports :**
  - **`Clk, Reset`** : Horloge et remise à zéro.
  - **`Start, Halt`** : Signaux de contrôle utilisateur.
  - **`JUMP`** : Entrée venant de l'ALU pour valider un saut conditionnel.
  - **`States`** : Signaux d'activation pour tous les autres modules (`Fetch_PC`, `Fetch_IR`, `Decode`, `Execute`, `Update_PC`).
  - **`BOOT`** : Signal maître basculant le système en mode lecture a travers le UART.
  - **`Code_len = PCi`** : Signal de fin de lecture du module Bootloader.



Fonctionnement

Le processus se décompose en 4 étape far :

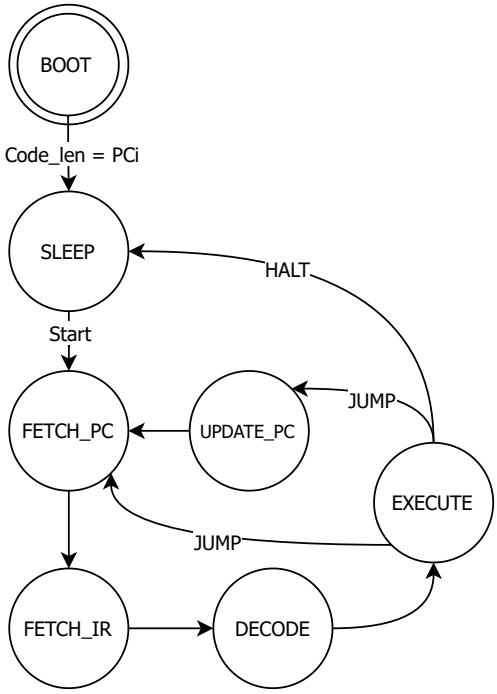
Fetch_PC, qui permet la validation de l'incrémentation de PC, et donc la récupération de l'instruction au niveau de la ROM.

Fetch_IR, est l'état d'activation du registre d'instruction, donc l'instruction est disponible au niveau du décodeur.

Decode, état "vide" pour laisser le temps au décodeur de séparer l'instruction en OPcode en Adresse, et activer le registre des adresse pour maintenir la valeur.

Execute, Maintien de la valeur en sortie de l'ALU/registre accumulateur.

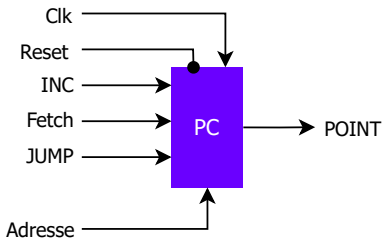
Graphe d'état



State	Explication
BOOT	Téléchargement de code a travers le port de communication UART
SLEEP	Attente d'une interruption via Start
FETCH_PC	Récupéré du numéro l'instruction a executé
FETCH_IR	Récupéré l'instruction et la transférer au decodeur
DECODE	Décoder l'instruction en OPcode pour l'ALU et Adresse pour la RAM
EXECUTE	Ordre pour REG Acc de maintenir la valeur Result check d'une instruction JUMP conditionel
UPDATE_PC	Incrémente PC (adresse de l'instruction) si JUMP = 0

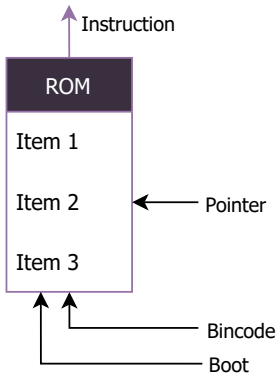
2. PC (Program Counter)

- **Fonction :** Compteur/Registre 12 bits qui contient l'adresse de l'instruction en cours ou à venir, la valeur est maintenu avec une impulsion sur Fetch.
- **Ports :**
  - **`Clk, Reset :`** Horloge et remise à zéro.
  - **`INC :`** Incrémente la valeur de +1.
  - **`JUMP :`** Charge directement une adresse (Saut).
  - **`Adresse (11:0) :`** Adresse cible en cas de JUMP.
  - **`POINT (15:0) :`** Vers le multiplexeur de la ROM. le pointeur d'adresse ROM en mode fonctionnement.
    Si JUMP = 1 , POINT reçoit adresse étendu sur 16bits donc "0000"&Adresse.

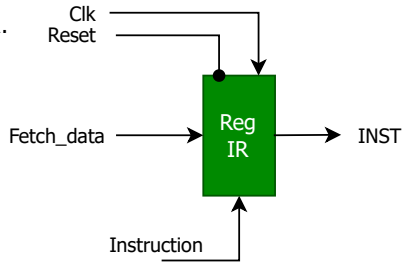


3. ROM (Mémoire d'Instructions)

- **Fonction :** Stocke le code binaire du programme. Elle est pilotée soit par le Bootloader (écriture), soit par le PC (lecture).
- **Taille :** 2^{16} = 65,5 kmots, un mot est codé sur deux octets = 131 koctets.
- **Ports:**
  - **`Pointer (15:0) :`** Adresse fournie par le MUX_TX (PC ou Bootloader).
  - **`Bincode (15:0) :`** Donnée venant de l'UART (en mode BOOT).
  - **`Boot (Write enable) :`** Activé uniquement en mode Boot.
  - **`Instruction (15:0) :`** L'instruction envoyée vers le registre IR.



4. Reg_IR (Instruction Register)



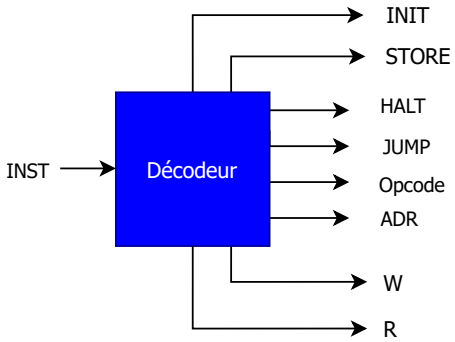
- **Fonction :** Capture et stabilise l'instruction lue dans la ROM pendant l'état `FETCH_IR`.
- **Ports :**
  - **`Clk, Reset :`** Horloge et remise à zéro.
  - **`Fetch_data :`** Activé par l'état `FETCH_IR`.
  - **`Instruction (15:0) :`** Vient de la ROM.
  - **`INST (15:0) :`** L'instruction vers le Décodeur.

L'instruction est codé de tel façon que en a un OPcode sure 4bit, et 12 bit pour les adresse :

OP3	OP2	OP1	OP0	ADR12	ADR11	ADR10	ADR9	ADR8	ADR7	ADR6	ADR5	ADR4	ADR3	ADR2	ADR1	ADR0

5. Décodeur d'Instructions

- **Fonction :** Sépare l'instruction en Adresse et Opcode qui active des signaux de commande pour tout le processeur.
- **Ports :**
  - **`INST (11:0) :`** Vient du Reg_IR.
  - **`INIT :`** Signal de sélection du MUX
    INIT = 1 pour cour circuité la RAM et donc initialiser le registre accumulateur en valeur immédiate.
  - **`STORE :`** Signal de sélection de mode pour la RAM.
  - **`HALT :`** Instruction d'arrêt et passage en état Sleep.
  - **`JUMP :`** Instruction de saut conditionnel.
  - **`Opcode (15:12) :`** Code de l'instruction en cours d'exécution.
  - **`ADR (11:0) :`** Adresse courante ou valeur immédiate (INIT).
  - **`W :`** Ordre d'écriture sur les pins du GPIO (Access via l'adresse xFFF).
  - **`R :`** Ordre de lecture sur les pins du GPIO (Access via l'adresse xFFE)

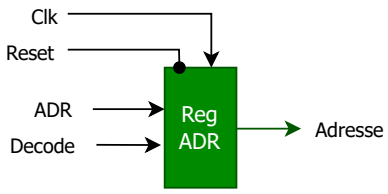


x	OPcode	Operation	Adresse	Signal	Effet
0	0000	INIT	ADR	INIT	Acc = "0000" & [Adresse]
1	0001	Load	xFFE	R	Acc = [valeur des pins de lecture]
2	0010	Store	xFFF	W	Pin de lecture = [Acc]
3	0011	JUMP	Adresse	JUMP	Pointer = [Adresse]
4	0100	Jz	Adresse	JUMP	Pointer = [Adresse] si [Acc] = x000
5	0101	Jn	Adresse	JUMP	Pointer = [Adresse] si Acc[15] = 1
6	0110	Jp	Adresse	JUMP	Pointer = [Adresse] si Acc[15] = 0
F	1111	HALT	x000	HALT	Quitte la boucle d'exécution pour aller en état Sleep

Particularité :
Pour lire des pins faut d'abord store la valeur du registre d'interface dans la case mémoire dédier puis charger la valeur
Store xFFE
Load xFFE

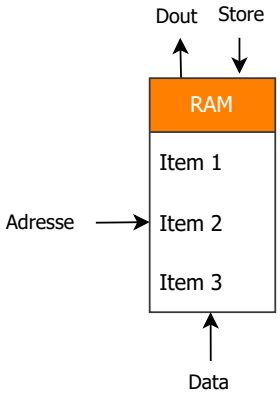
6. Reg_Adresse (ADR)

- **Fonction :** Maintient l'adresse stable pour la RAM pendant toute la durée du cycle décodage.
- **Ports :**
  - **`Clk`, `Reset` :** Horloge et remise à zéro.
  - **Decode :** Activé pendant l'état `DECODE`.
  - **`ADR (11:0)` :** Vient du Reg_IR.
  - **`Adresse (11:0)` :** Vers le bus d'adresse de la RAM, PC et MUX (Pour la gestion des valeur immédiate avec INIT).



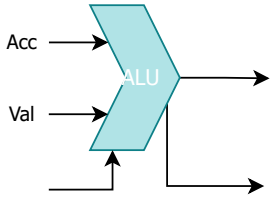
7. RAM (Mémoire de Données)

- **Fonction :** Zone de stockage pour les variables du programme et l'interface GPIO.
- **Taille :** $2^{16-4} = 2^{12}$ = 4 kmots, un mot est codé sur deux octets = 8 koctets.
- **Ports :**
  - **`Adresse (11:0)` :** Vient du Reg_ADR.
  - **`Data (15:0)` :** Vient de l'Accumulateur ou des Input Pins (via `MUX_RX`).
  - **`Store` :** Signal `STORE` venant du décodeur pour passer en mode écriture.
  - **`Dout (15:0)` :** Vers le MUX, selon l'instruction (autres que INIT) vers l'ALU.



8. ALU (Unité Arithmétique et Logique)

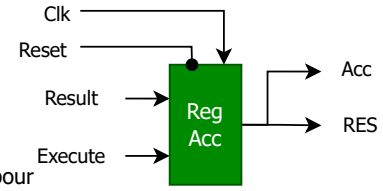
- **Fonction :** Réalise les 9 calculs (ADD, SUB, MULT, etc.) restant non effectué par le Décodeur et autres systèmes.
- **Ports :**
  - **`Acc (15:0)` :** Entrée venant de l'Accumulateur.
  - **`Val (15:0)` :** Entrée venant de la RAM ou d'une valeur immédiate.
  - **`Opcode (3:0)` :** Code de l'opération à effectuer.
  - **`Result (15:0)` :** Vers le registre Accumulateur.
  - **znp :** Flags d'état (Zéro, Négatif, Positif).



x	OPcode	Operation	Adresse	Operation
7	0111	Add	Adresse	Result = RAM[Adresse] + [Acc]
8	1000	Substarct	Adresse	Result = RAM[Adresse] - [Acc]
9	1001	Multiplication	Adresse	Result = RAM[Adresse] * [Acc]
A	1010	AND	Adresse	Result = RAM[Adresse] AND Bitwise [Acc]
B	1011	NOT	Adresse	Result = NOT Bitwise [Acc]
C	1100	Xor	Adresse	Result = RAM[Adresse] Xor Bitwise [Acc]
D	1101	Shitf_left	x000	Result = Acc[14:0] & '0'
E	1110	Shitf_right	x000	Result = '0' & Acc[15:1]

9. Reg_ACC (Accumulateur)

- **Fonction :** Registre de travail principal qui stocke le résultat de l'ALU ou les chargements RAM.
- **Ports :**
  - **Execute :** Activé lors de l'état `EXECUTE pour maintenir le resultat`.
  - **`Result (15:0)` :** Vient du résultat de calcul de l'ALU.
  - **`RES (15:0)` :** Sortie principale (Résultat), réinjectée dans l'ALU (Acc) et accessible pour le `STORE`.



10. Module GPIO (Interface Entrées/Sorties)

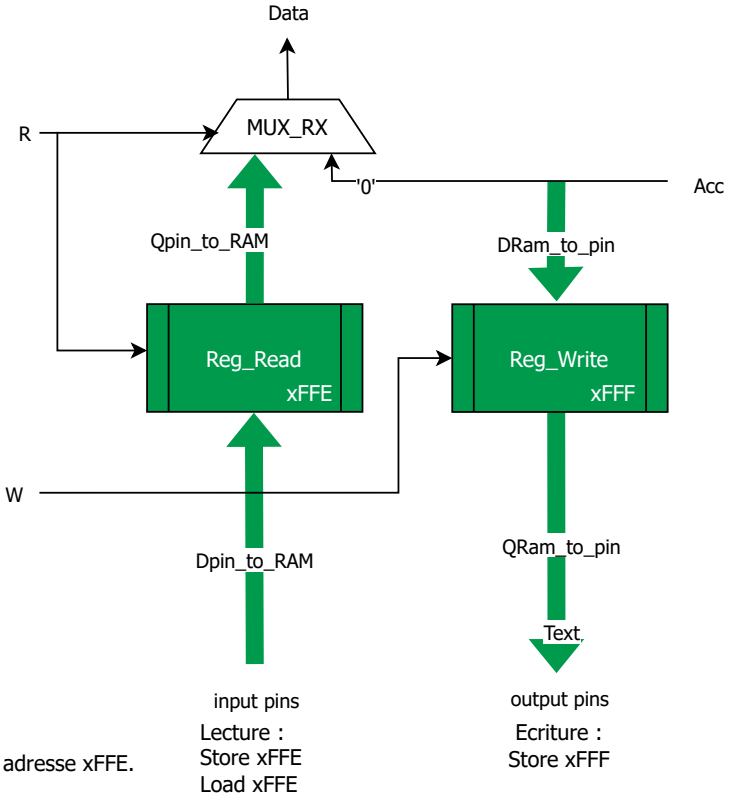
Le module **GPIO** permet au processeur BOLT-16 de communiquer avec son environnement physique (switches, boutons, LEDs). Plutôt que d'utiliser des instructions d'entrée/sortie spécifiques, le GPIO est intégré directement dans l'espace adressable de la mémoire aux adresses réservées **`xFFE`** (Lecture) et **`xFFF`** (Écriture).

Fonctionnement

L'architecture un système pour stabilisé les données sur les pins:

Lecture (Input) - Adresse `xFFE` : Pour lire l'état des pins, l'utilisateur doit d'abord exécuter une instruction `Store xFFE`. Cette commande déclenche le signal interne `R`, qui capture l'état des `input pins` dans le registre d'interface `Reg_Read`. La valeur est stockée en RAM et peut ensuite être récupérée dans l'accumulateur via un `Load xFFE`.

Écriture (Output) - Adresse `xFFF` : Pour piloter des sorties, l'utilisateur exécute un `Store xFFF`. Le décodeur active alors le signal `W`, en plus de sauvegarder la valeur il transfère la valeur actuelle de l'accumulateur vers le registre `Reg_Write`, qui maintient la valeur en sortie des pins.



Ports :

- **`Dpin_to_RAM (15:0)` :** Valeur lu sur les pins.
- **`Qpin_to_RAM (15:0)` :** Valeur transmise a la RAM adresse xFFE.
- **`QRam_to_pin (15:0)` :** Valeur a écrite sur les pins.
- **`DRam_to_pin (15:0)` :** Valeur a écrire, venant de Acc.
- **`R` :** Instruction de lecture sur les pins (Store xFFE et Load xFFE).
- **`W` :** Instruction d'ecriture sur les pins (Store xFFF).
- **`Acc` :** Registre de sortie.
- **`Data` :** Valeur lu sur les pins et maintenu par le registre Reg_Read.

Registres d'interface dédiés

`Reg_Read` (`xFFE`) : Registre qui "gèle" la valeur des entrées au moment du `STORE xFFE, pour ensuite pouvoir lire sa valeur via un Load xFFE.`

`Reg_Write` (`xFFF`) : Registre de maintien des données de sortie, Store xFFF,.